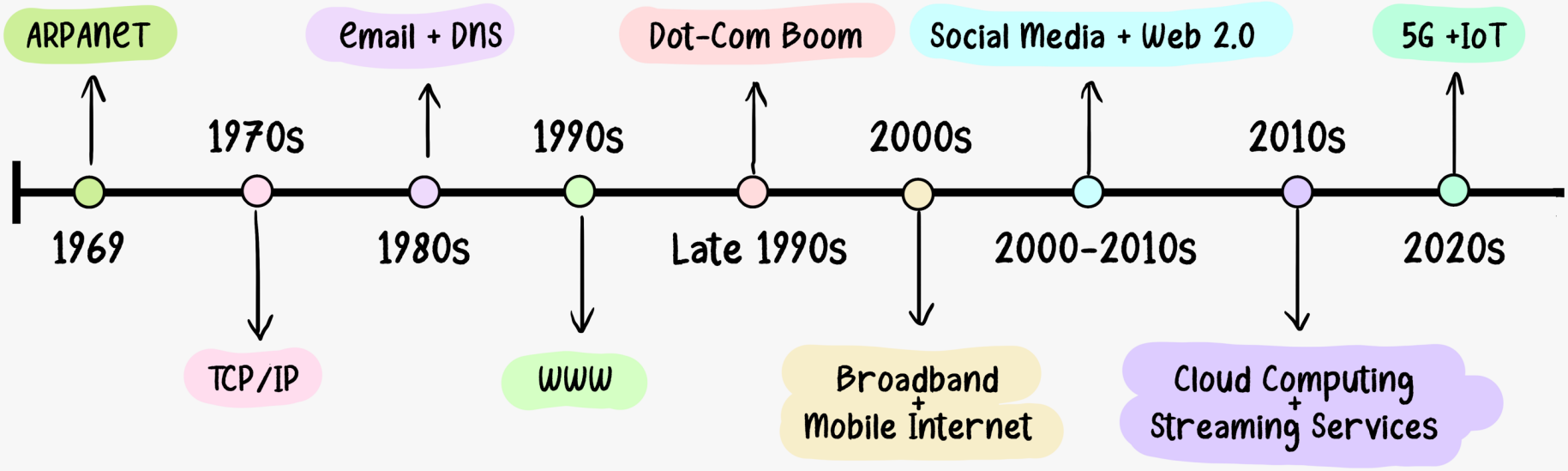
Networking and Internetworking: basic issues for distributed computing

Agenda

- Types of networks
- Network principles
- Internet protocols
- Case studies

Internet Evolution Timeline



APR 2026

ESSENTIAL DIGITAL HEADLINES

OVERVIEW OF THE ADOPTION AND USE OF CONNECTED DEVICES AND SERVICES

https://datareportal.com/global-digital-overview



GLOBAL OVERVIEW

TOTAL POPULATION	UNIQUE MOBILE PHONE SUBSCRIBERS	INDIVIDUALS USING THE INTERNET	SOCIAL MEDIA USER IDENTITIES	MONTHLY ACTIVE GENERATIVE AI USERS
8.28 BILLION	**5.83 BILLION**	**6.12 BILLION**	**5.79 BILLION**	**2.42 BILLION**
URBANISATION	vs. POPULATION	vs. POPULATION	vs. POPULATION	vs. POPULATION
80.5%	**70.4%**	**73.8%**	**69.9%**	**29.2%**

SOURCES: MANOCHI ANALYSIS; UNITED NATIONS; GOVERNMENT RESOURCES; GSMA INTELLIGENCE; ITU; EUROSTAT; CNNIC; SIMILARWEB; PLATFORM RESOURCES; COMPANY EARNINGS REPORTS; OCDH; BETA RESEARCH CENTER. **ADVISORY:** SOCIAL MEDIA USER IDENTITIES AND GENERATIVE AI USERS MAY NOT REPRESENT UNIQUE INDIVIDUALS. **NOTES:** WHERE AVAILABLE, THE FIGURE FOR URBANISATION REPRESENTS THE U.N.'S REPORTED VALUE FOR THE PERCENTAGE OF THE TOTAL POPULATION LIVING IN TOWNS AND CITIES. FIGURES FOR GENERATIVE AI USE ONLY INCLUDE THE USE OF STANDALONE GENERATIVE AI PLATFORMS, AND DO NOT FACTOR THE USE OF AI FUNCTIONALITY WITHIN OTHER SOFTWARE OR APPLICATIONS. **COMPARABILITY:** BASE REVISIONS; SOURCE CHANGES. SEE NOTES ON DATA.

we are social

MANOCHI

Principles of networking

- The principles on which computer networks are based include: protocol layering, packet switching, routing, and data streaming.
- Internetworking techniques enable heterogeneous networks to be integrated.
- The Internet is the major example; its protocols are almost universally used in distributed systems.
- The addressing and routing schemes used in the Internet have withstood the impact of its enormous growth.
- They have been revisionated to accommodate the growth of users and usages and to meet new application requirements for mobility, security and quality of service

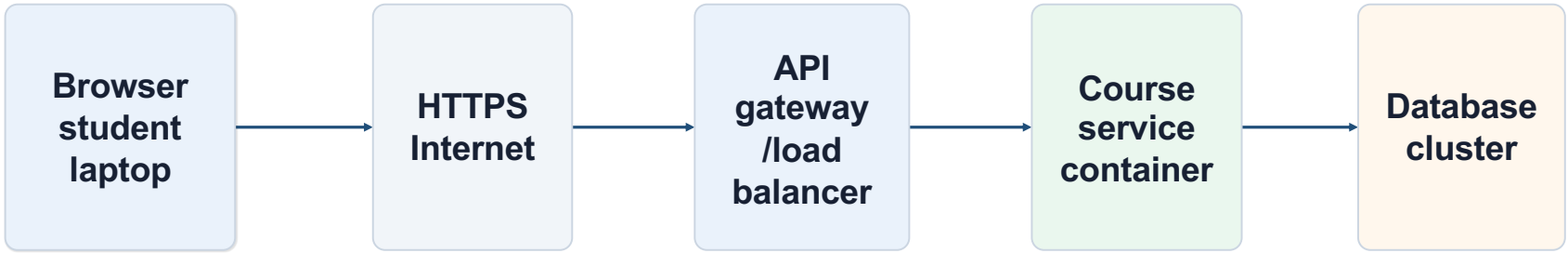
- Principles of networking are counter-intuitive for software developers with limited experience of networks

Distributed systems are based on networks

- Distributed systems are made of software (processes) which use communication infrastructures:
  local area networks, wide area networks, and internetworks.
- The performance, reliability, scalability, mobility, and Quality of Service properties of the underlying networks impact the design and behaviour of distributed systems.
- Technological innovations have resulted in the emergence of wireless networks and high-performance networks with Quality of Service guarantees

Running example: a grade request

We will reuse one request to connect network facts to distributed-system design decisions.



GET /grades?course=DSS

What can go wrong?

Latency accumulates across hops; bandwidth limits payloads; packets can be lost; topology and endpoints can change.

Design consequence

Do not design the software as if a remote call were a local function call.

Common **false** assumptions on networks

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one global administrator

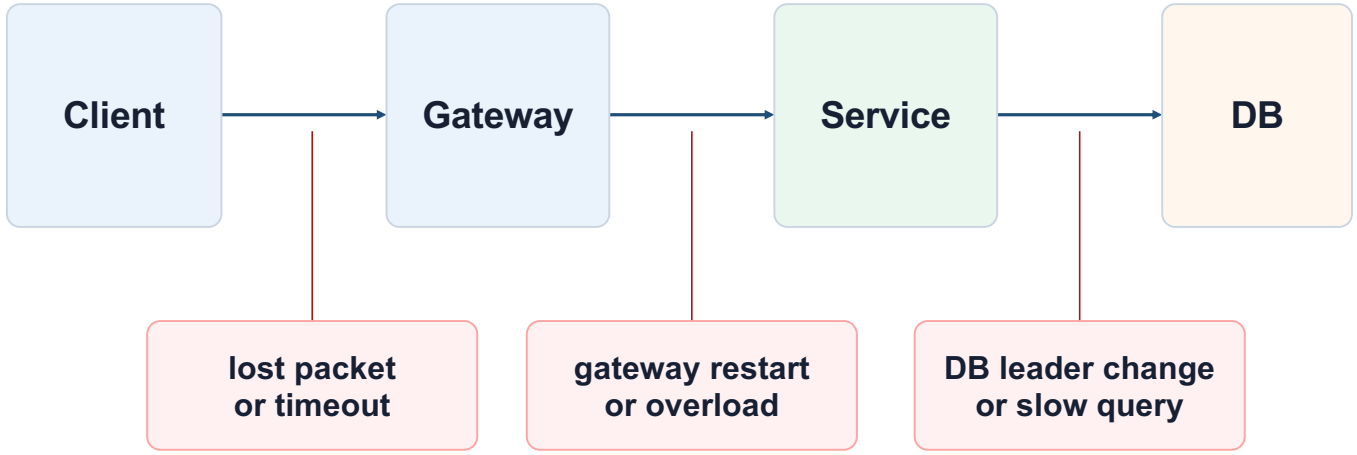
•When developing non-distributed applications, most of these issues will likely not show up

Consequences of the fallacies

Fallacy	Effect
The network is unreliable	App needs error handling/retry lost messages
Latency is greater than zero	App should minimize # of requests
Bandwidth is finite	App should send small payloads
The network is unsecure	App must secure its data/authenticate requests
Topology does change	Changes affect latency, bandwidth and endpoints
There is not one administrator	Changes affect ability to reach destination
Transport cost is more than zero	Costs must be budgeted, someone has to pay
The network is heterogeneous	Affects reliability, latency and bandwidth

Reliability: every arrow can fail

The browser sees one request; the system sees many partial failures.



Implementation pattern

Set timeouts. Retry only safe or idempotent operations. Use correlation IDs to trace one user request across several components.

Distributed-systems lesson

A failure may be local, remote, intermediate, or only observed as silence. Error handling is part of the protocol design.

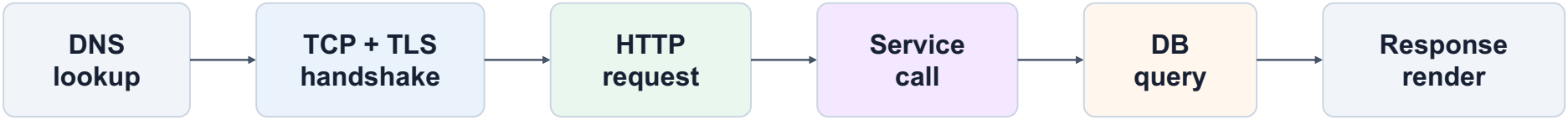
Measuring Quality of Service: four main network performance metrics

- **Bandwidth**: The speed of a link: bits per second
- **Latency (delay)**: The time it takes for a packet to go from its source to its end destination: measured in milliseconds
- **Loss**: The amount of packet loss, which typically occurs due to network congestion: measured by percentile of data loss
- **Jitter**: The irregular speed of packets as a result of congestion, which can result in packets arriving late and out of sequence; the average of the absolute differences between the expected arrival time of each packet and its actual arrival time: measured in milliseconds

- Example: Packet loss should be no more than 1%, and network latency shouldn't exceed 150 ms one-way (300 ms return). Jitter should be below 30ms.

Latency budget: the request is a chain

A small page interaction may trigger several network round trips.



Network transfer time = latency + message length / data transfer rate

Programming rule

Minimize remote round trips before optimizing local computation. Cache stable data and batch independent reads where correctness permits.

Student observation

The same service feels different on campus LAN, home Wi-Fi, and mobile networks because the path and contention differ.

Network performance main parameters

•The primary network performance parameters are the **latency** and the **point-to-point data transfer rate**:

- **Latency** is the delay that occurs after a send operation is executed and before data starts to arrive at the destination computer. It can be measured as the time required to transfer an empty message (we are considering only network latency, which forms a part of the process-to-process latency).
- **Data transfer rate** is the speed at which data can be transferred between two computers in the network once transmission has begun, usually quoted in bits per second.

•The time required for a network to transfer a message containing *length* bits between two computers is:

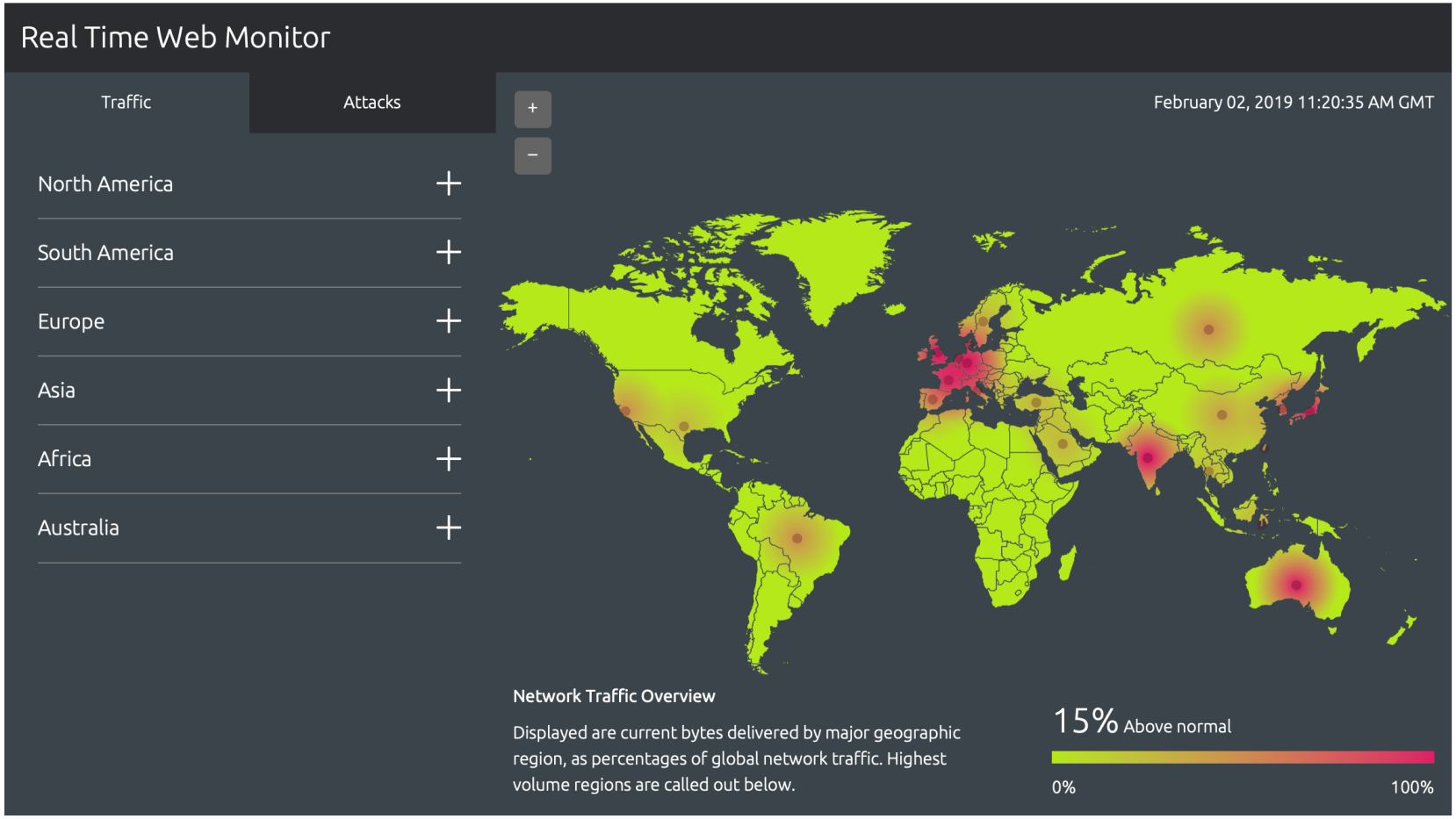
- *Message transmission time = latency + length ⁄ data transfer rate*

Network parameters in the Internet

- On the Internet, round-trip latencies are in the 5–500 ms range, with means of 20–200 ms depending on distance
- Requests transmitted across the Internet are 10–100 times slower than those sent on fast local networks.
- The bulk of this time difference derives from switching delays at routers and contention for network circuits.

Try also https://globe.akamai.com

www.akamai.com



InterProcess Communication (IPC)

Question: how do processes on different machines exchange information?

•Answer: with difficulty … ☹

Established computer network facilities are **too primitive**, resulting in distributed software too difficult to develop – a new model is required

- IPC is the “heart” of every distributed system.
- Four IPC models are popular:

  RPC; RMI; Streams; MOM (msg oriented middleware)

Communication choices: what changes in the example?

Choose the transport semantics that match the interaction, not the fashion of the month.

HTTPS / HTTP/2
Browser API calls
Reliable byte stream

gRPC over HTTP/2
Service-to-service RPC
Schema + streaming

Message broker
Async notification
Decouple sender/receiver

UDP / QUIC cases
Low latency, loss-tolerant or app-managed reliability

In the grade-request example

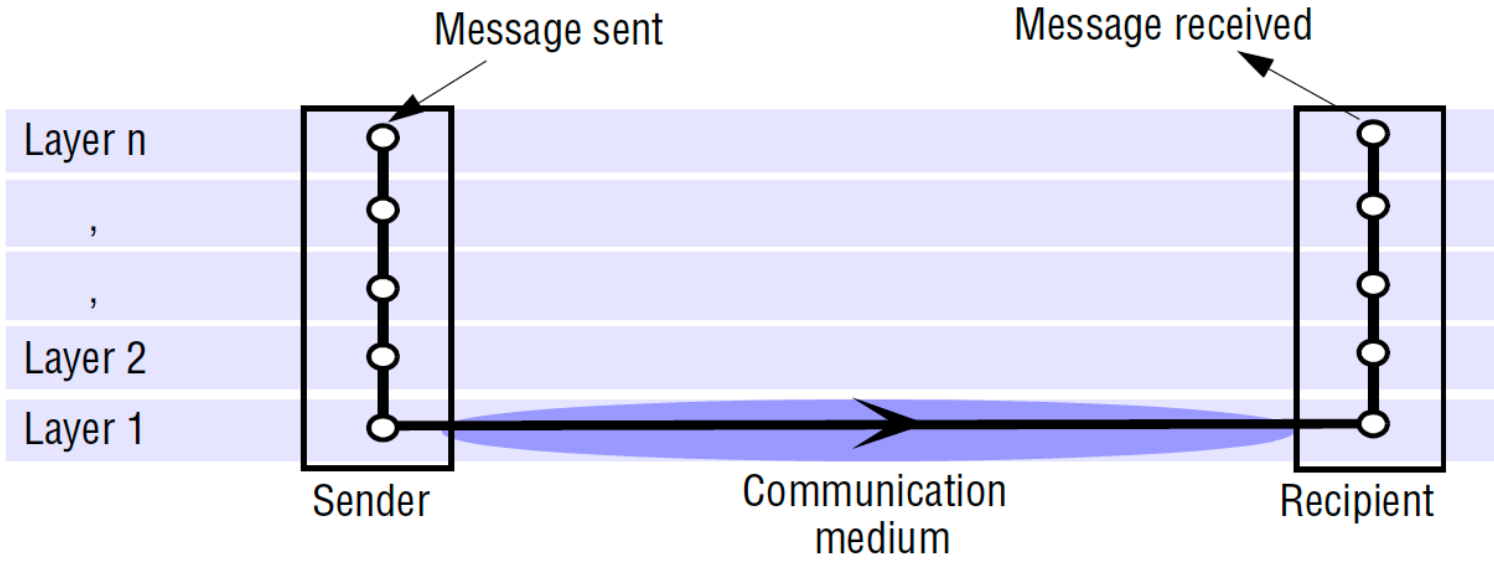
The user-facing read normally benefits from reliable, secure request/response communication. Long-running updates can be pushed or polled. Audit events can be sent asynchronously.

What students should explain

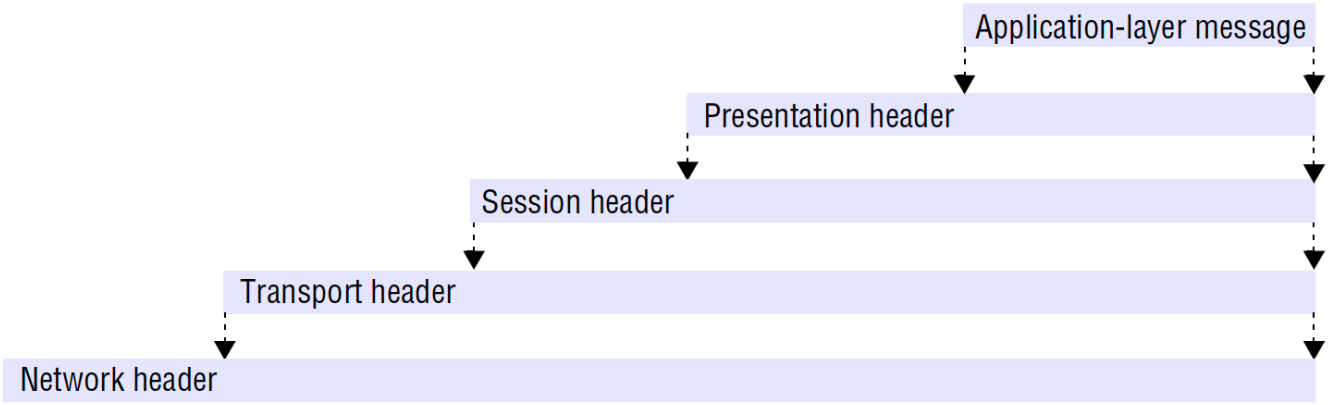
For each protocol choice, state what it assumes about ordering, loss, retries, security, message size, and user-visible latency.

Mini-exercise: redesign /grades for a weak mobile network. Which calls do you remove, cache, batch, retry, or make asynchronous?

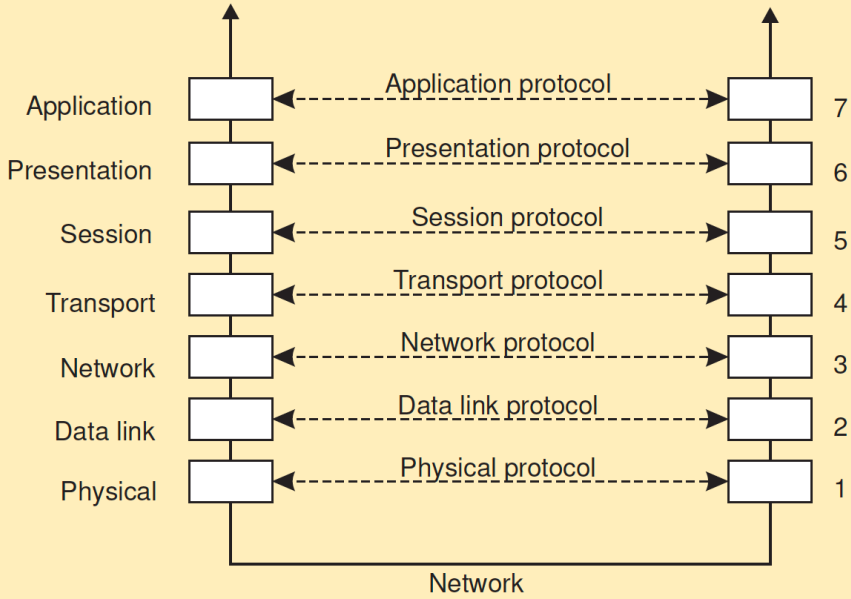
Conceptual layering of protocol software



Encapsulation as it is applied in layered protocols



Basic networking model



Drawbacks

- Focus on message-passing only
- Often unneeded or unwanted functionality
- Violates access transparency

Bandwidth and packetization: payload size matters

The same API can be cheap or expensive depending on the representation it sends.

Bad response

Full student profile
all courses
all attempts
large JSON

Better response

Only current course
needed fields
paginated history
compressed body

Streaming case

Log tail or video
partial results
back-pressure
QoS visible

Network fact

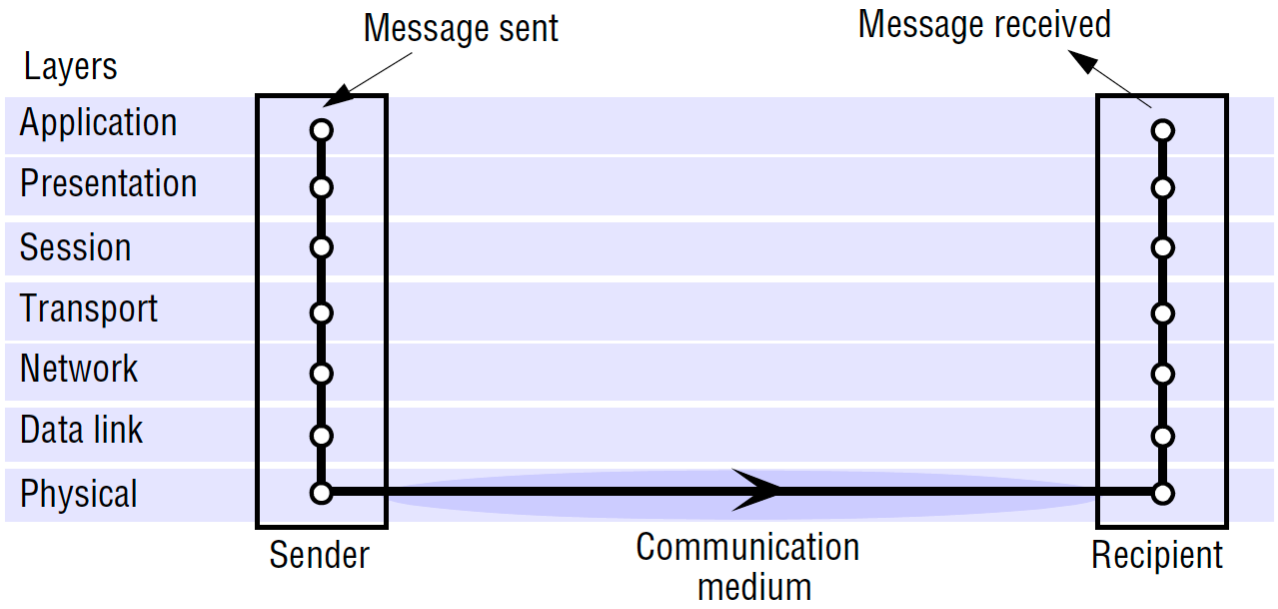
Finite bandwidth means that large payloads amplify latency and compete with other traffic.

MTU reminder

If a message exceeds the network maximum transfer unit, it must be split into multiple packets and reassembled.

Design question for the example: should /grades return everything now, or only what the UI can display immediately?

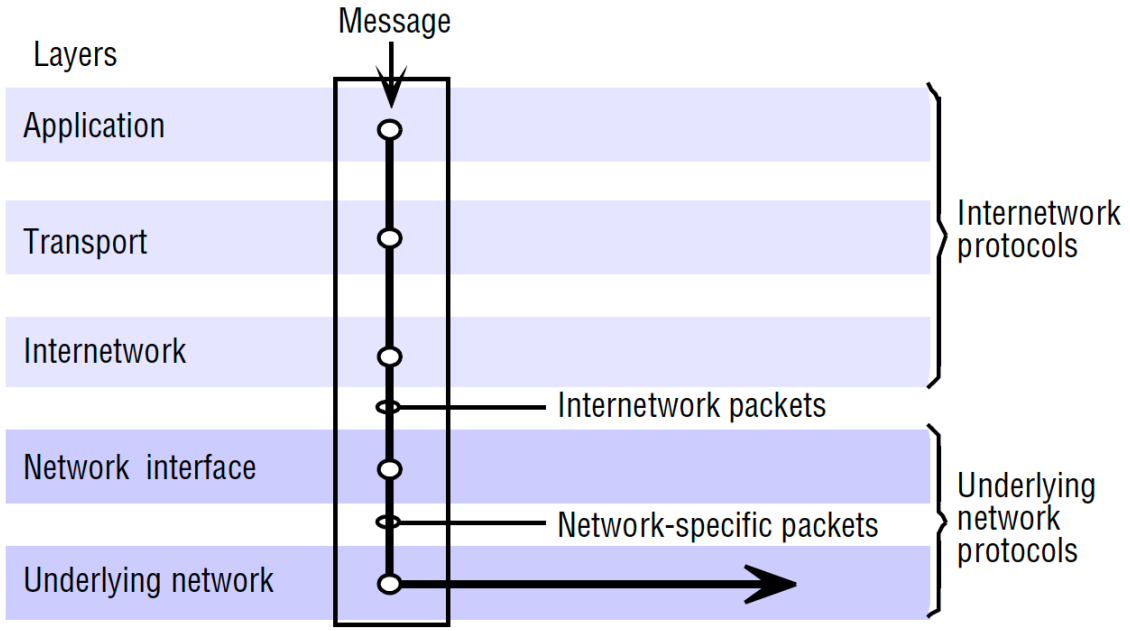
Protocol layers in the ISO Open Systems Interconnection (OSI) model



OSI protocol summary

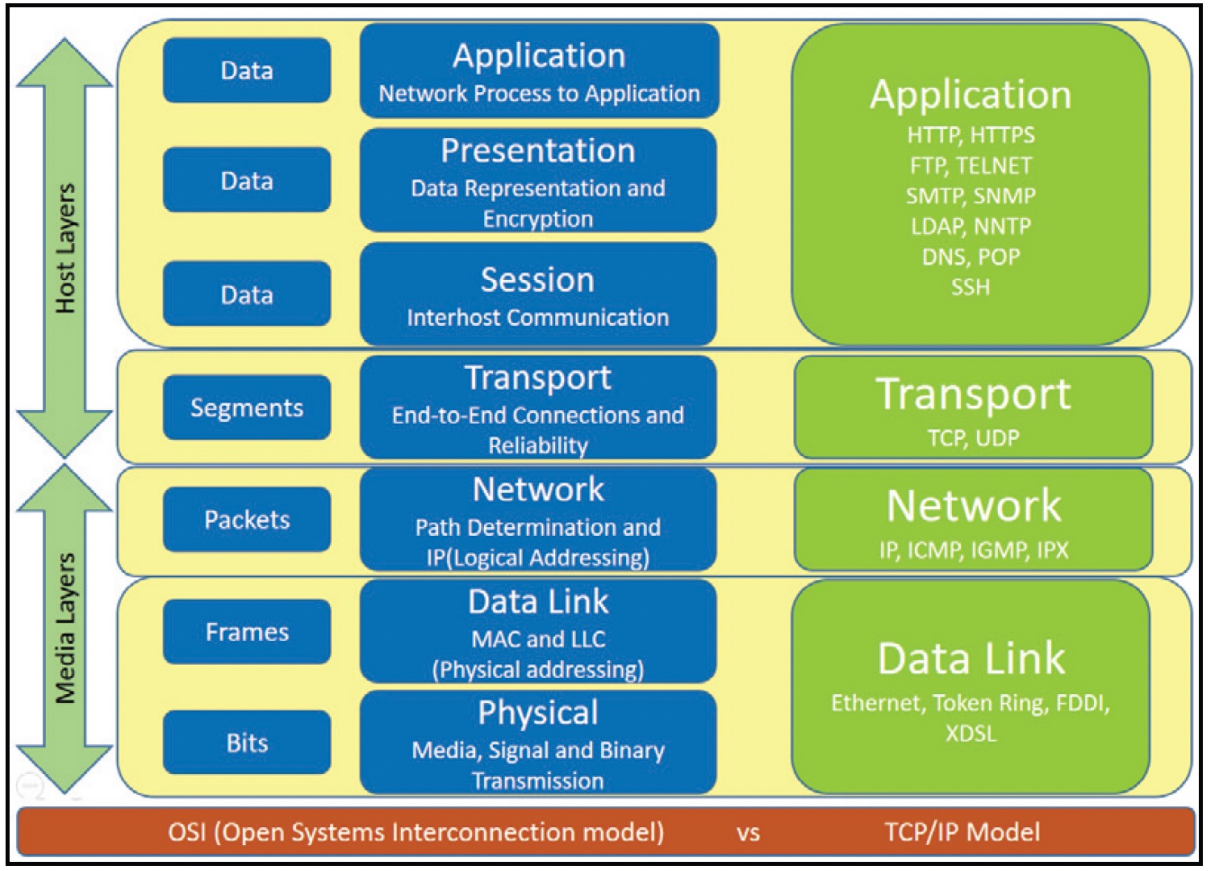
Layer	*Description*	*Examples*
Application	Protocols that are designed to meet the communication requirements of specific applications, often defining the interface to a service.	HTTP, FTP , SMTP, CORBA IIOP
Presentation	Protocols at this level transmit data in a network representation that is independent of the representations used in individual computers, which may differ. Encryption is also performed in this layer, if required.	Secure Sockets (SSL),CORBA Data Rep.
Session	At this level reliability and adaptation are performed, such as detection of failures and automatic recovery.	
Transport	This is the lowest level at which messages (rather than packets) are handled. Messages are addressed to communication ports attached to processes, Protocols in this layer may be connection-oriented or connectionless.	TCP, UDP
Network	Transfers data packets between computers in a specific network. In a WAN or an internetwork this involves the generation of a route passing through routers. In a single LAN no routing is required.	IP, ATM virtual circuits
Data link	Responsible for transmission of packets between nodes that are directly connected by a physical link. In a WAN transmission is between pairs of routers or between routers and hosts. In a LAN it is between any pair of hosts.	Ethernet MAC, ATM cell transfer, PPP
Physical	The circuits and hardware that drive the network. It transmits sequences of binary data by analogue signalling, using amplitude or frequency modulation of electrical signals (on cable circuits), light signals (on fibre optic circuits) or other electromagnetic signals (on radio and microwave circuits).	Ethernet base- band signalling, ISDN

Internetwork layers



For many distributed systems, the lowest-level interface is that of the network layer

Clustering the stack: the TCP/IP model



Conclusions

- We have summarized the networking principles that are needed as a basis for programming distributed systems (approaching them from the point of view of a distributed system programmer).
- Packet networks and layered protocols provide the basis for communication in distributed systems.
- Local area networks are based on packet broadcasting on a shared medium; Ethernet is the dominant technology
- Wide area networks are based on packet switching to route packets to their destinations through a connected network
- Routing is a key mechanism and a variety of routing algorithms are used, of which the distance-vector method is the most basic but effective.
- Congestion control is needed to prevent overflow of buffers at the receiver and at intermediate nodes.

Networks vs distributed systems

- A computer network is an interconnected collection of independent computers able to exchange messages
- A computer network usually requires users to login to a machine, explicitly submit jobs remotely, explicitly move files/data around the network

- In a distributed system the existence of multiple computers is transparent to the user
- The (distributed) operating system automatically allocates jobs to processors & moves files without explicit user intervention

Exercise

•Compare connectionless (UDP) and connection-oriented (TCP) communication for the implementation of each of the following application-level or presentation-level protocols:

i. virtual terminal access (for example: Telnet);
ii. file transfer (for example: FTP);
iii. user location (for example: rwho or finger);
iv. information browsing (for example: HTTP);
v. remote procedure call.

Answer

i. The long duration of sessions, the need for reliability and the unstructured sequences of characters transmitted make connection-oriented communication most suitable for this application. Performance is not critical in this application, so the overheads are of little consequence.

ii. File calls for the transmission of large volumes of data. Connectionless would be ok if error rates are low and the messages can be large, but in the Internet, these requirements aren't met, so TCP is used.

iii. Connectionless is preferable, since messages are short, and a single message is sufficient for each transaction.

iv. Either mode could be used. The volume of data transferred on each transaction can be quite large, so TCP is used in practice.

v. RPC achieves reliability by means of timeouts and re-trys. So connectionless (UDP) communication is often preferred.